

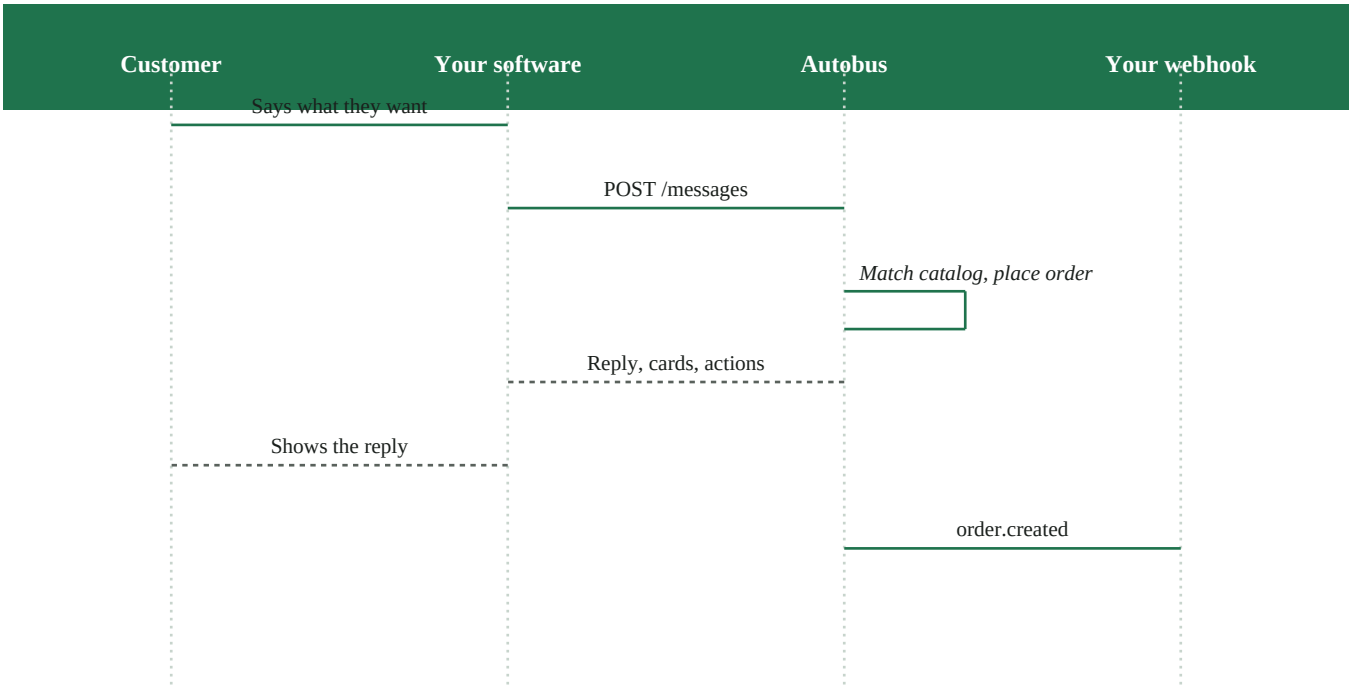
# Embedded chat

Autobus answers your customers and places orders. Your software owns the screen. You send each message to Autobus and show the reply. When an order is placed, Autobus also calls a webhook on your server so you can fulfill it.

Base URL: `https://<your-autobus-host>/api/v1/embed`

## How a turn works

The customer talks to your product. Your product asks Autobus. Autobus answers from the catalog and, when the customer confirms, creates the order.



### Chat

Your app calls Autobus

One message in, one reply out

### Catalog

Your app pushes items

Only when mode is synced

### Webhook

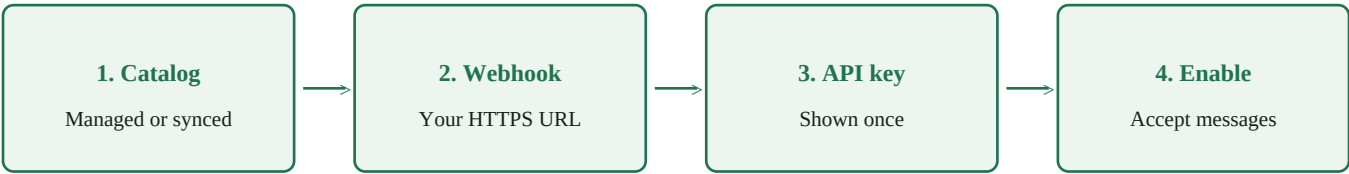
Autobus calls your URL

Order, handoff, sync failure

| Direction    | Who starts it | What it is for                                         |
|--------------|---------------|--------------------------------------------------------|
| Chat         | Your software | One customer message in, one reply out                 |
| Catalog sync | Your software | Push inventory Autobus is allowed to sell              |
| Webhook      | Autobus       | Order placed, order updated, or a person was requested |

## 1. Turn it on in the portal

In the Autobus app, open Settings, then Embedded chat.



Managed in Autobus means products are entered in the portal, so you skip catalog sync. Synced from your software means your system is the source of truth and you push items with PUT /catalog.

The full API key is shown once. Store it as a secret. The portal keeps only a hash. Copy the webhook signing secret from the same screen and use it to verify that a webhook came from Autobus. The key is scoped to that business.

```
Authorization: Bearer ab_live_...
X-API-Key: ab_live_... (also accepted)
```

2. Catalog

Push this only when catalog mode is synced. A managed catalog returns HTTP 409. The bot sells only these rows. Do not attach a product list to the chat message.

PUT /api/v1/embed/catalog

```
{
  "items": [{
    "external_id": "SKU-RICE-5",
    "kind": "product",
    "name": "Rice Bag 5kg",
    "description": "Local perfumed rice",
    "price": { "amount": "75.00", "currency": "GHS" },
    "stock": { "tracked": true, "quantity": 40 },
    "category": "Grocery",
    "active": true
  }, {
    "external_id": "SVC-DELIVERY",
    "kind": "service",
    "name": "Same-day delivery",
    "price": { "amount": "20.00", "currency": "GHS" },
    "stock": { "tracked": false },
    "active": true
  }
]
```

| Field          | Required     | Meaning                                                                     |
|----------------|--------------|-----------------------------------------------------------------------------|
| external_id    | yes          | Your SKU. Later upserts with the same id overwrite the item.                |
| kind           | no           | product (default) or service.                                               |
| name           | yes          | Name the bot is allowed to say and sell.                                    |
| price.amount   | no           | Decimal string. Defaults to 0. The bot quotes this price.                   |
| price.currency | no           | Display hint. Orders use the business currency in Autobus.                  |
| stock.tracked  | no           | true for a counted product. false for a service that can always be ordered. |
| stock.quantity | when tracked | Integer on hand. 0 means the bot will not sell it.                          |
| active         | no           | false hides the item from the bot and keeps order history.                  |

Send changes as they happen, and a full snapshot at least once a day. Up to 200 items per request. GET /api/v1/embed/catalog returns the items Autobus currently has for this key.

### 3. Send a message

## POST /api/v1/embed/messages

```
{
  "conversation_id": "thread_441",
  "customer": {
    "external_id": "cust_8841",
    "name": "Ama Mensah",
    "phone": "+233201234567"
  },
  "message": { "id": "msg_91", "text": "I want 2 bags of rice" }
}
```

| Field                | Required | Meaning                                                                                             |
|----------------------|----------|-----------------------------------------------------------------------------------------------------|
| customer.external_id | yes      | Stable id in your software. Combined with the business, this is the conversation key.               |
| customer.name        | no       | Pre-filled onto the order when the customer has not typed a name.                                   |
| customer.phone       | no       | Pre-filled onto the order. The bot asks when a phone is still needed.                               |
| conversation_id      | no       | Your thread id. Copied onto the order and the webhook.                                              |
| message.id           | yes      | Your id for this turn. The same id returns the original response and does not place a second order. |
| message.text         | yes      | What the customer said.                                                                             |

## Response

```
{
  "conversation_id": "thread_441",
  "reply": {
    "text": "Order placed for 2 x Rice Bag 5kg ...",
    "products": [{ "external_id": "SKU-RICE-5", "name": "Rice Bag 5kg", "price": "75.00", "currency": "GHS", "stock_status": "in stock" }],
  },
  "actions": [{
    "type": "order.created",
    "order": {
      "order_number": "ORD-20260925-10421",
      "source": "embed",
      "external_customer_id": "cust_8841",
      "items": [{ "external_id": "SKU-RICE-5", "name": "Rice Bag 5kg", "quantity": 2, "unit_price": "75.00" }],
      "total": "150.00",
      "currency": "GHS",
      "status": "pending",
      "payment_status": "pending"
    }
  ]
}
```

Show reply.text to the customer. Render reply.products as cards when you want. Treat actions as the machine result. actions is empty when the bot only answered. conversation.handoff is added when the customer asks for a person and handoff is enabled. Payment stays in your software. The order is created unpaid and also appears in the Autobus portal.

## 4. Receive webhooks

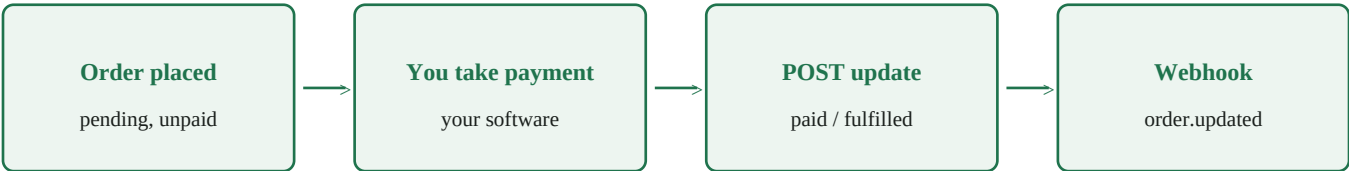
Autobus POSTs JSON to the URL you saved.

```
{
  "id": "evt...",
  "type": "order.created",
  "created_at": "2026-09-25T00:40:00Z",
  "data": { }
}
```

| Header              | Meaning                                                                    |
|---------------------|----------------------------------------------------------------------------|
| X-Autobus-Event     | order.created, order.updated, conversation.handoff, or catalog.sync_failed |
| X-Autobus-Signature | sha256= plus HMAC-SHA256 of the raw body, using your webhook secret        |

Verify the signature against the raw bytes before you parse JSON. Reject anything that does not match. order.created and order.updated put the order object in data. catalog.sync\_failed puts failed items and an upserted count in data. Respond with HTTP 2xx. Autobus does not retry a failed delivery in this version, so act on the synchronous actions array immediately and use the webhook as the copy your backend stores.

5. Update an order



POST /api/v1/embed/orders/{order\_number}

```
{
  "payment_status": "paid",
  "fulfillment_status": "fulfilled",
  "payment_reference": "TXN-12345"
}
```

payment\_status: pending, paid, partial, refunded, failed. fulfillment\_status: unfulfilled, partial, fulfilled, shipped, delivered. order\_status: pending, processing, confirmed, cancelled, completed.

Errors

| HTTP | When                                                   |
|------|--------------------------------------------------------|
| 401  | Missing or unknown API key, or embedded chat is off    |
| 400  | A required chat or catalog field is missing or invalid |
| 404  | Order number is not on this business                   |
| 409  | Catalog push while mode is still managed in the portal |

Minimal loop

- 1 Store message.id, then POST /messages with the API key.
- 2 If actions contains order.created, record order\_number and each item external\_id, then fulfill.
- 3 When payment completes, POST the order update.
- 4 If catalog mode is synced, PUT /catalog when inventory changes.